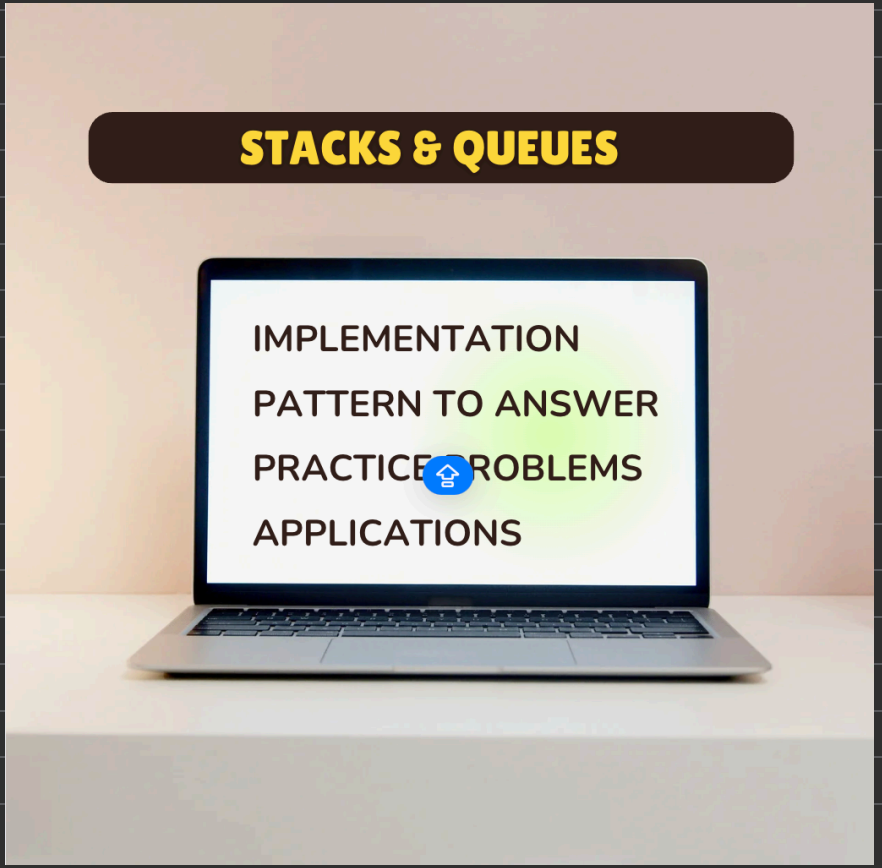
WAP to find Sum of all Subset

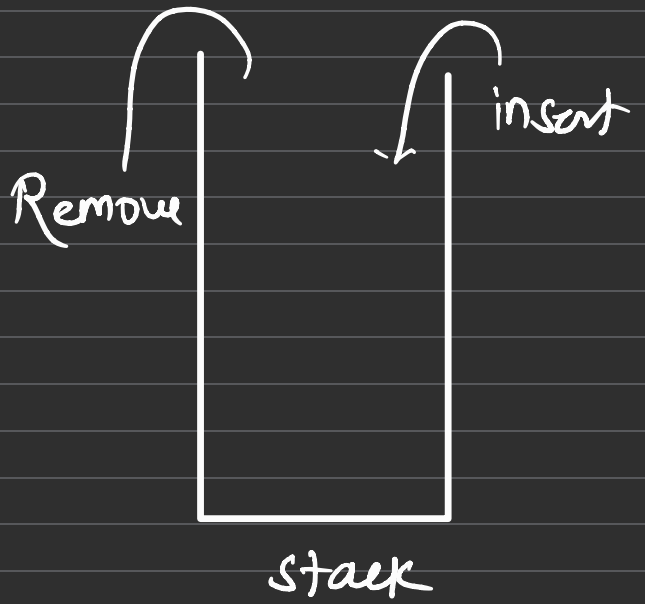
🎯 खुल जाएंगे सभी रास्ते, रुकावटों से लड़ तो सही..
सब होगा हासिल, तू जिद्द पर अड़ तो सही...



Stack

Stack is linear data structure, in which element is inserted from TOP of stack & removed from TOP of stack.

It mainly follows, LIFO (Last In First Out) or FILO (First In Last Out) order.



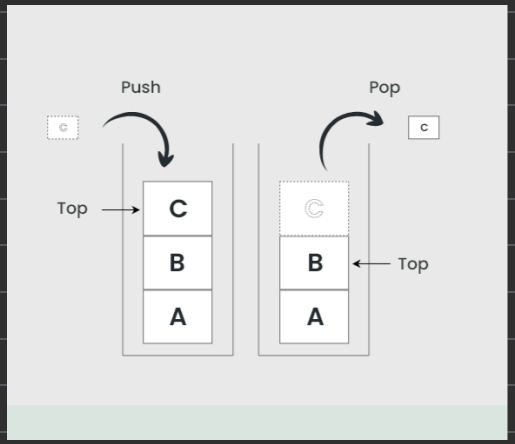
Operations on Stack

Push: To insert an element.

Pop: To remove an top element.

Top: Returns the topmost element.

Is Empty: Tells whether stack is empty or not.



* All the operations are performed in O(1)

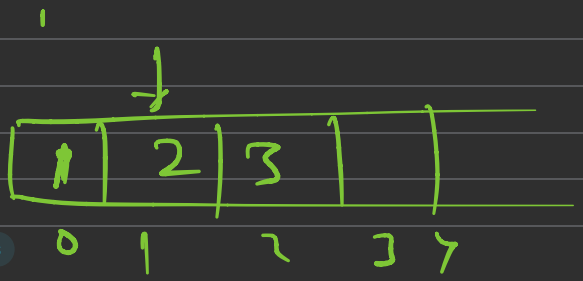
push(10) push(20) push(30) pop Top

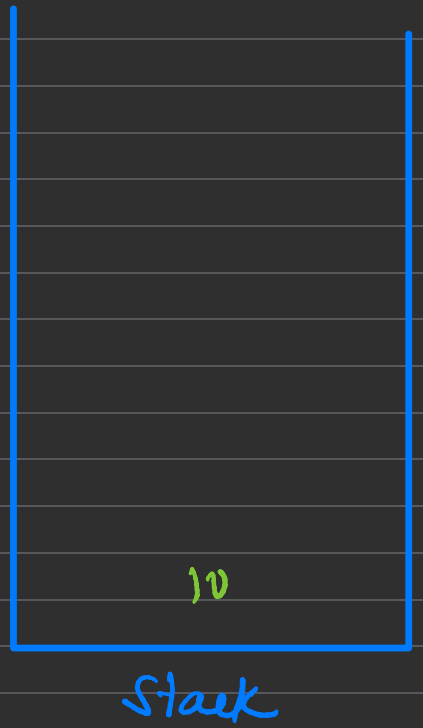
isEmpty pop pop isEmpty

Output: 20

false

true





Applications on Stack

1. Reverse the String
2. Infix to Postfix
3. Prefix to Postfix
4. Balanced Parenthesis
5. Recursion Activation Stack Record

Stack in STL

Header file: include <bits/stdc++.h>

```
stack <int> st;

// push
st.push(10); // Push at top of stack

// pop
st.pop(); // remove top element

// top
st.top(); // returns top element

// is emtpy
st.empty(); // check if stack is empty
```

Stack in JAVA Collections

```
ArrayDeque <Integer> st = new ArrayDeque<>();

// push
st.push( e: 10); // Push at top of stack

// pop
st.pop(); // remove top element

// top
st.peek(); // returns top element

// is emtpy
st.isEmpty(); // check if stack is empty
```

Implementing Stack

```
void push ( int data)
{
    if (top == N-1)
    {  overflow
    }
    ++top
    st[top] = data
}
```

Implementing Stack

pop
(Remove top element)

```
void pop()
{
    if (top == -1)
    {
        underflow
    }
    top = top - 1
}
```

Peek()

if (top == -1) return

print (st(top))

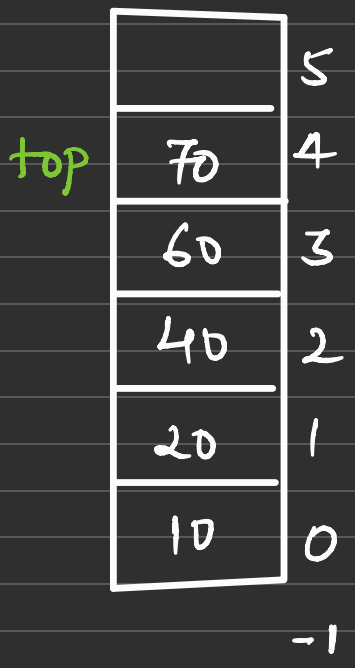
Empty()

if (top == -1) return true

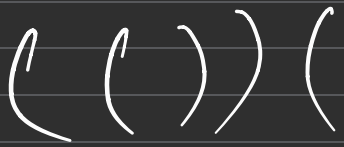
return false

```
void traverse ()
{
    if (top == -1)
        return

    for (i=0 ; i<=top ; i++)
    {
        print (st[i])
    }
}
```



Problems on Stacks



20. Valid Parentheses

Solved

Easy | Topics | Companies | Hint | Re-do

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

> **Input:** `s = "()"`
>
> **Output:** `true`

Example 2:

> **Input:** `s = "()[]{}"`
>
> **Output:** `true`

Example 3:

> **Input:** `s = "(]"`

(( ) [ ]) }

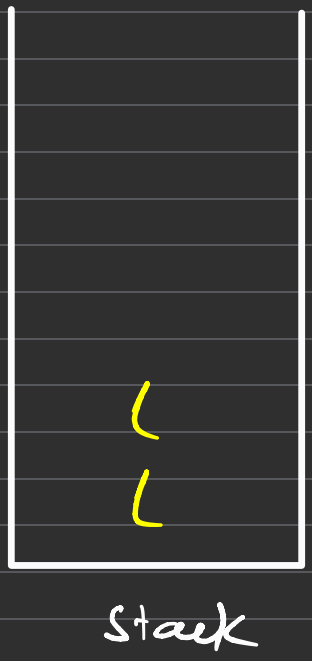
if open bracket
- push in st

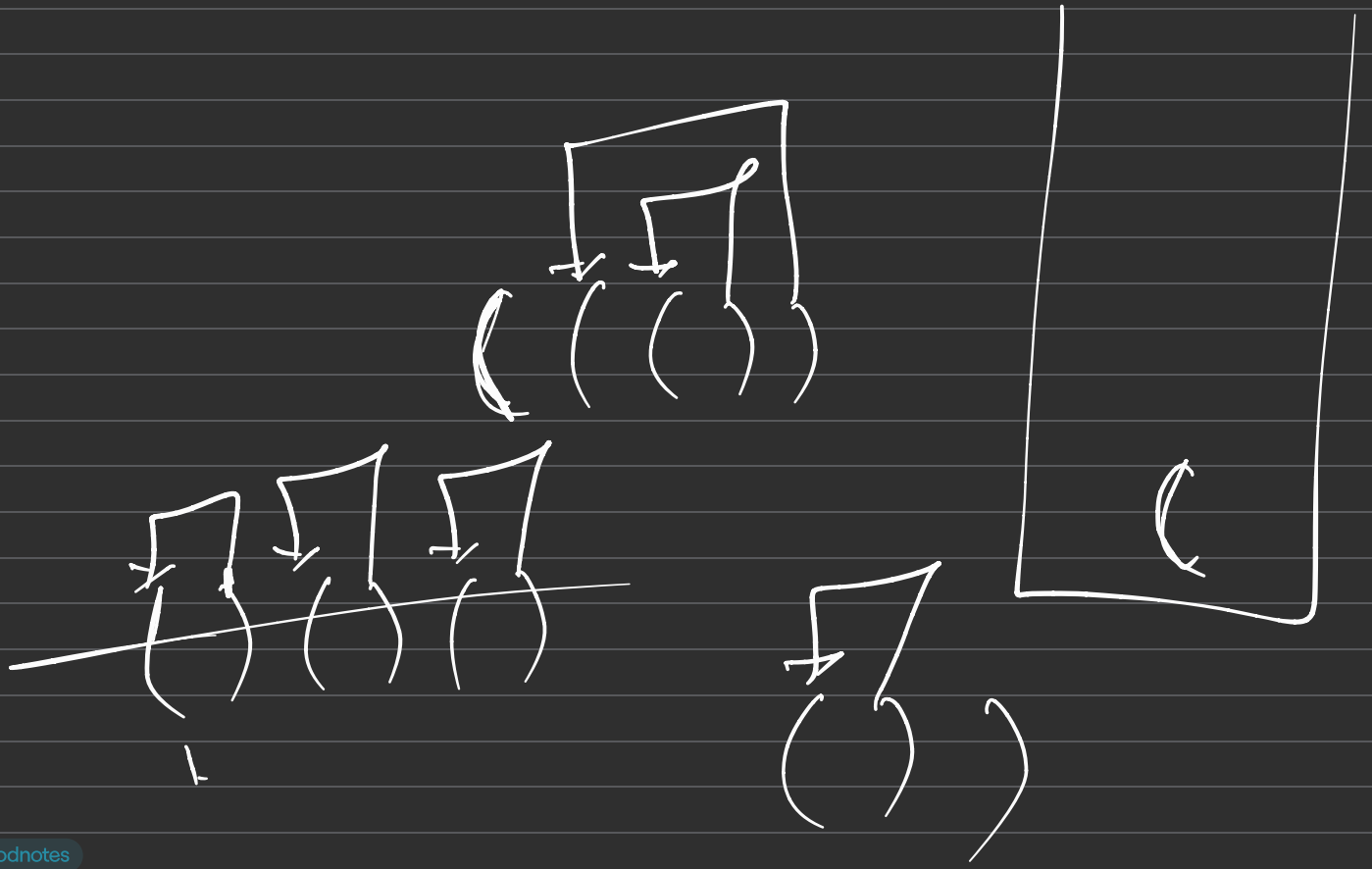
if closing bracket

check (st.top(), s[i])

top s(i)

( ) [ ]) }





```
bool isBalanced (string s)
{   stack <char> st

    for (i=0 ; i<n ; i++)
    {
        if (s[i] == '(' || s[i] == '{' || s[i] == '[')
        {       st.push (s[i])
        }
        else {      if (ischek (st.top(), s[i])
                        st.pop()
                    else
                        return false
        }   }

    return st.empty()
}
```

Monotonic Stack

Next Greater Element

Difficulty: **Medium** Accuracy: **32.95%** Submissions: **464K+** Points: **4** Average Time: **20m**

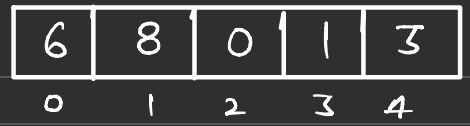
Given an array **arr[ ]** of integers, the task is to find the next greater element for each element of the array in order of their appearance in the array. Next greater element of an element in the array is the nearest element on the right which is greater than the current element.
If there does not exist next greater of current element, then next greater element for current element is -1. For example, next greater of the last element is always -1.

Examples

Input: arr[] = [1, 3, 2, 4]
Output: [3, 4, 4, -1]
Explanation: The next larger element to 1 is 3, 3 is 4, 2 is 4 and for 4, since it doesn't exist, it is -1.

Input: arr[] = [6, 8, 0, 1, 3]
Output: [8, -1, 1, 3, -1]
Explanation: The next larger element to 6 is 8, for 8 there is no larger elements hence it is -1, for 0 it is 1 , for 1 it is 3 and then for 3 there is no larger element on right and hence -1.

```
for ( i=0 ; i<n ; i++)        ( N )
{
    for ( j = i+1 ; j<n ; j++)   ( N-1 )
    {
        if ( a[i] < a[j])
        {
            print ( a[j])
            break
        }
    }
    if (j == N)
        print (-1)
}
```



Stack

6	8	0	1	3
0	1	2	3	4

nums

cur

top

$8 \geq 0$

8 NYE → -1

```
vector<int> NGE(arr, N)
{
    vector<int> ans
    stack<int> st
    for (i = N-1 ; i>=0 ; i--)
    {
        if (st.empty())
        {
            push (a(i))
        }
        else {
            while ( ! st.empty() && a(i) >= top()
            {
                st. pop
            }
```

ans.push(-1)

```
if (st.empty())
    ans. push(-1)
else
    ans.push (top())
```

} }

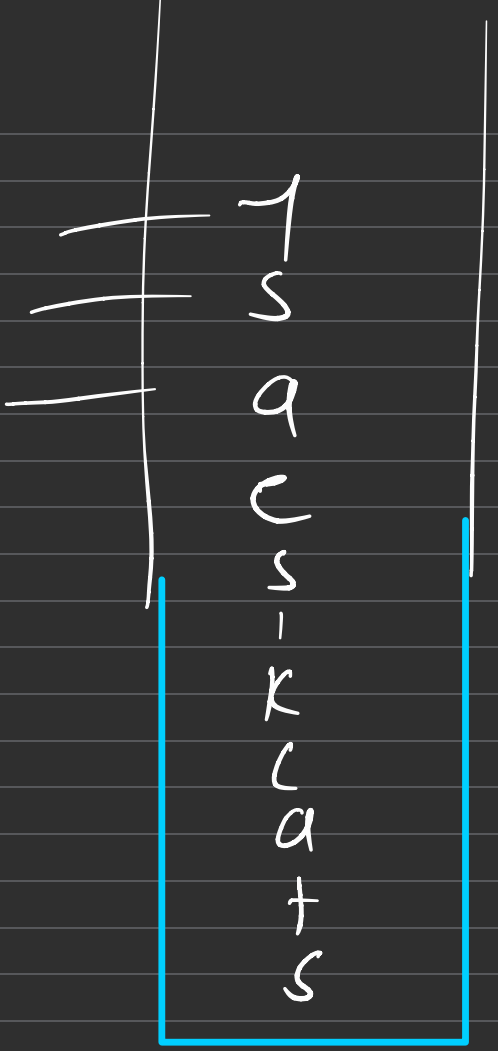
Reverse the string using Stack

You are given a string, print it in reverse order.

string: stackiseasy

output : ysaesikcats

ysaesikcats



* infix

a + b

(between)

operator = + | - | * | / | %

Operand = a b

* prefix

+ ab

(before)

postfix

ab +

after

Eq: $A+B+C-(D*E)/F$

$*\ /\ \%$ ①

$+\ -$ ②

postfix:

$A+B+C-(DE*)/F$

$A+B+C-(DE*F)/$

$(AB+)+C-(DE*F)/$

$(AB+C+)-(DE*F)/$

$(AB+C+)(DE*F)/-$

$A+B+(D*E)/F$

$*\ \div\ /$ ①

$+\ -$ ②

postfix

$A+B+(DE*)/F$

$A+B+(DE*F/)$

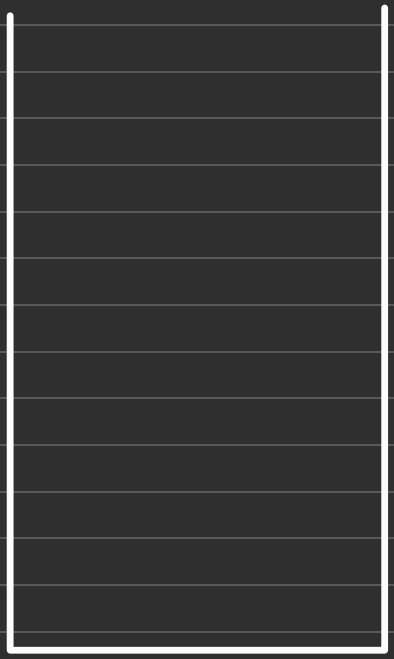
$A+B(DE*F/)+$

$ABDE*F/++$

A + B + D * F / E

* / ÷ ①

\+ − ②



A B + D F * E / +